

ArchIntent — Combined System Testing Flow

Prepared for: DawoodKhan72

2026-04-18

Contents

1	Overview	3
2	Preflight: Start Services	3
3	Phase 1: Database Reset, Seed, and Verify	3
3.1	Reset and seed	3
3.2	Verify tables (14 tables)	4
3.3	Verify seeded users (7 users)	4
3.4	Verify key columns	4
4	Phase 2: Authentication and Role Routing (Browser)	4
4.1	Wrong-role redirect test	5
4.2	Pending account test	5
5	Phase 3: Public Browse & Portfolio (Not logged in)	5
5.1	Architect listing	5
5.2	Architect profile and portfolio projects	5
5.3	Project details	5
6	Phase 4: Messaging from Browse (Client)	5
7	Phase 5: Project Creation & Manual Matching	6
7.1	Create project (client)	6
7.2	Manual match insertion (internal API)	6
7.3	Select architect	6
8	Phase 6: Chat Testing (Real-time)	6
8.1	Before/after selection	6
8.2	Real-time verification	7
8.3	Unread badge	7
9	Phase 7: Agreement + Signatures	7

10 Phase 8: Payment (Stripe Test)	7
11.1 Architect uploads design	7
11.2 Client reviews, requests revision	8
11.3 Approve & release payment	8
11.4 Auto-approval (scheduler) test (optional)	8
12 Phase 10: Architect Review	8
13 Phase 11: Construction Bidding with Budz	9
13.1 Client posts job	9
13.2 Contractor bids	9
13.3 Second contractor bid	9
13.4 Client accepts top bid	9
14 Phase 12: Contractor Portfolio View in Bid	9
15 Phase 13: Complete Project + Contractor Review	9
16 Phase 14: Admin Testing	9
16.1 Dashboard stats	10
16.2 User management	10
16.3 Verify pending architect	10
16.4 Analytics and logs	10
17 Phase 15: Security & API Authorization (Postman)	10
17.1 Role isolation	10
17.2 API authorization checks	10
17.3 Upload security	10
17.4 Token revocation test	10
18 If Something Breaks (Standard Recovery)	11
19 Quick Reference — Accounts	11

1 Overview

This document combines the provided testing guides into a single, structured flow you can follow to validate ArchIntent end-to-end:

- Environment & seeding verification
- Authentication & role isolation
- Public browsing (architect listings/portfolio)
- Project creation & manual matching
- Messaging (real-time)
- Agreement + signatures
- Payment (Stripe test)
- Design delivery & approval (incl. revision and auto-approval)
- Construction bidding with Budz
- Reviews (architect + contractor)
- Admin operations (verify/suspend/analytics/logs)
- Security checks (API authorization, uploads, token revocation)

2 Preflight: Start Services

Open 3 terminals:

1. Backend:

```
php artisan serve
```

2. Reverb (real-time chat):

```
php artisan reverb:start
```

3. Frontend:

```
npm run dev
```

Expected: Backend on <http://127.0.0.1:8000> and frontend on <http://localhost:5173>.

3 Phase 1: Database Reset, Seed, and Verify

3.1 Reset and seed

1. Reset DB:

```
php artisan migrate:fresh
```

2. Seed:

```
php artisan db:seed --class=TestDataSeeder
```

3.2 Verify tables (14 tables)

In MySQL:

```
SHOW TABLES;
```

Expected: Exactly these 14 tables:

- users, architects, portfolios, portfolio_images
- contractors, contractor_portfolios, contractor_portfolio_images
- projects, project_matches, bids
- agreements, agreement_signatures, payments, admin_logs

3.3 Verify seeded users (7 users)

Run:

```
SELECT email, role, account_status FROM users;
```

Expected: You should see:

```
admin@test.com      admin      active
client@test.com     client     active
architect@test.com  architect  active
contractor@test.com contractor active
architect2@test.com architect  active
contractor2@test.com contractor active
pending@test.com    architect  pending
```

3.4 Verify key columns

DB Check: Run:

```
DESCRIBE users;
DESCRIBE projects;
DESCRIBE payments;
```

Expected: Confirm ENUMs/columns match your spec (roles, account statuses, project statuses, payment statuses, and mda_verification_deadline).

4 Phase 2: Authentication and Role Routing (Browser)

Open four tabs and login:

Tab	Login	Expected Redirect
1	admin@test.com / Test@1234	/dashboard/admin
2	client@test.com / Test@1234	/dashboard/client
3	architect@test.com / Test@1234	/dashboard/architect

4	contractor@test.com / Test@1234	/dashboard/contractor
---	------------------------------------	-----------------------

4.1 Wrong-role redirect test

In client tab (Tab 2), manually visit:

`http://localhost:5173/dashboard/architect`

Expected: Redirect back to `/dashboard/client`.

4.2 Pending account test

In an incognito window, login:

`pending@test.com / Test@1234`

Expected: Show `/pending-verification` and not the dashboard.

5 Phase 3: Public Browse & Portfolio (Not logged in)

Open incognito (logged out).

5.1 Architect listing

Visit:

`http://localhost:5173/architects`

Expected: Show verified architects (e.g., Sara Malik, Bilal Ahmed). **Do not show** pending (e.g., Zara Shah).

5.2 Architect profile and portfolio projects

Click Sara Malik card:

`/architect/{id}`

Expected: Projects grid shows ARC-0001, ARC-0002, ARC-0003, each with a `project_ref` badge.

5.3 Project details

Click *View Project Details* on ARC-0001:

`/architect/{id}/project/ARC-0001`

Expected: Full details page; reference ARC-0001 prominent.

6 Phase 4: Messaging from Browse (Client)

In Tab 2 (client logged in), go to `/architects`. On Sara Malik card click *Message*. Send:

Hi Sara, I love your work on ARC-0001

Expected: Navigate to `/messages` and see conversation.

7 Phase 5: Project Creation & Manual Matching

7.1 Create project (client)

In Tab 2, visit:

`/dashboard/client/create-project`

Fill the form (title/type/location/budget/description) and submit.

Expected: Redirect to `/project/{id}`; status `created`; progress stepper at Step 1.

7.2 Manual match insertion (internal API)

Get IDs:

```
SELECT project_id FROM projects ORDER BY project_id DESC LIMIT 1;
SELECT architect_id FROM architects WHERE user_id = 3; -- adjust if needed
SELECT architect_project_id FROM architect_projects WHERE project_ref='ARC-0001';
```

Then POST (Postman/Thunder Client):

POST `http://127.0.0.1:8000/api/internal/project-matches`

Headers:

X-Internal-Key: `<APP_INTERNAL_KEY from .env>`

Content-Type: `application/json`

Body:

```
{
  "project_id": 1,
  "matches": [
    { "architect_id": 1, "portfolio_id": 1, "match_score": 91.50 }
  ]
}
```

Expected: Success response; project status becomes `matched`.

7.3 Select architect

Refresh `/project/1`:

- **Expected:** Status shows `matched`
- **Expected:** *View Matched Architects* visible

Go to `/project/1/matches`, then click *Select This Architect*. **Expected:** Redirect to `/project/1`; status `architect_selected`.

8 Phase 6: Chat Testing (Real-time)

8.1 Before/after selection

Client (Tab 2) on `/project/1`: click *Message Architect* and send:

Hi Sara, you have been selected for my project!

Architect (Tab 3) on `/messages`: reply:

Thank you Ahmed! I will start creating the agreement.

8.2 Real-time verification

Keep both tabs open on `/messages`. **Expected:** Messages appear instantly without refresh.

8.3 Unread badge

Client: leave messages page, go to `/dashboard/client`. **Expected:** Navbar shows unread badge.
Return to `/messages`. **Expected:** Badge clears.

9 Phase 7: Agreement + Signatures

Architect (Tab 3): `/project/1` → *Create Agreement* → `/project/1/agreement`.

Fill all fields and:

1. Save as Draft **Expected:** draft saved
2. Finalize for Signing **Expected:** status `pending_signatures`, read-only view appears
3. Architect signs (name + checkbox) **Expected:** architect signed

Client (Tab 2): `/project/1/agreement` sign as client. **Expected:** Fully executed banner + *Make Payment* button.

DB Check::

```
SELECT COUNT(*) FROM agreement_signatures WHERE agreement_id = 1;  
SELECT project_status FROM projects WHERE project_id = 1;
```

Expected: signatures = 2; project_status = `payment_pending`.

10 Phase 8: Payment (Stripe Test)

Client (Tab 2): click *Make Payment* → `/project/1/payment`.

Use test card:

- 4242 4242 4242 4242
- Exp: 12/29
- CVC: 123

Expected: Spinner + disabled button; success screen; return to project.

DB Check::

```
SELECT payment_status, stripe_payment_id FROM payments WHERE project_id = 1;  
SELECT project_status FROM projects WHERE project_id = 1;
```

Expected: payment_status=`held`; stripe_payment_id**begins with** pi; project_status = `design_in_progress`.

11 Phase 9: Design Delivery + Revision + Approval

11.1 Architect uploads design

Architect (Tab 3): `/project/1` upload any PDF. **Expected:** Progress bar + success toast; status `design_delivered`.

DB Check::

```
SELECT design_file_path, design_delivered_at, mda_verification_deadline, project_status
FROM projects WHERE project_id = 1;
```

Expected: deadline = delivered_at + 3 days.

11.2 Client reviews, requests revision

Client: refresh /project/1. **Expected:** Teal review box + countdown timer + Download button.

Click *Request Revision*, submit revision text. **Expected:** Status returns to design_in_progress; architect upload returns.

11.3 Approve & release payment

After re-upload, client clicks *Approve Design and Release Payment*. **Expected:** Status design_approved; *Post Construction Job* visible.

DB Check::

```
SELECT project_status FROM projects WHERE project_id = 1;
SELECT payment_status FROM payments WHERE project_id = 1;
```

Expected: project_status=design_approved; payment_status=completed.

11.4 Auto-approval (scheduler) test (optional)

Set deadline in past:

```
UPDATE projects
SET mda_verification_deadline = DATE_SUB(NOW(), INTERVAL 1 HOUR)
WHERE project_id = 1 AND project_status = 'design_delivered';
```

Run:

```
php artisan schedule:run or php artisan app:auto-approve-designs
```

Expected: Project auto-approves.

12 Phase 10: Architect Review

Client on /project/1 (design approved):

- Leave review (5 stars, title, body)
- **Expected:** review locks (no edit/delete)

Verify on public architect profile:

- **Expected:** average rating updates
- **Expected:** review count and breakdown correct

13 Phase 11: Construction Bidding with Budz

13.1 Client posts job

Client: click *Post Construction Job*. **Expected:** status construction_open.

13.2 Contractor bids

Contractor (Tab 4): /construction-jobs → View & Bid. Submit bid with Budz slider (e.g., 30).

DB Check::

```
SELECT budz_spent FROM bids ORDER BY bid_id DESC LIMIT 1;
SELECT balance FROM budz_wallets WHERE contractor_id = 1;
```

13.3 Second contractor bid

Login as contractor2@test.com and bid with Budz=10.

13.4 Client accepts top bid

Client: on /project/1, confirm ordering (30 above 10), accept Hassan. **Expected:** Accepted/Rejected badges; status contractor_selected.

DB Check::

```
SELECT bid_status FROM bids;
SELECT balance FROM budz_wallets WHERE contractor_id = 2;
```

Expected: first accepted, second rejected; contractor2 refund applied per your rules.

14 Phase 12: Contractor Portfolio View in Bid

Client: expand *View Our Past Work* in accepted bid. **Expected:** Horizontal portfolio scroll with CON-0001, CON-0002 badges.

Click CON-0001 details:

/contractor/{id}/project/CON-0001

Expected: project reference displayed prominently; message button visible.

15 Phase 13: Complete Project + Contractor Review

Manually set completed:

```
UPDATE projects SET project_status = 'completed' WHERE project_id = 1;
```

Client refresh /project/1: **Expected:** Contractor review card appears. Submit (e.g., 4 stars). **Expected:** Contractor rating updates on public profile/bid views.

16 Phase 14: Admin Testing

Admin (Tab 1):

16.1 Dashboard stats

Expected: Total Users=7, Active Projects=1, Total Revenue=PKR 2,500,000.

16.2 User management

Suspend client@test.com. **Expected:** Client gets logged out or sees suspended message on refresh. Re-activate; client can log in again.

16.3 Verify pending architect

Admin approves pending (e.g., pending@test.com / Zara Shah). **Expected:** Pending removed; user now goes to /dashboard/architect.

16.4 Analytics and logs

Visit:

- /admin/analytics **Expected:** charts load and reflect test data
- /admin/logs **Expected:** actions recorded (verify, suspend, payment release, etc.)

17 Phase 15: Security & API Authorization (Postman)

17.1 Role isolation

Confirm client cannot open architect/admin dashboards, contractor cannot open client dashboard, etc.

17.2 API authorization checks

Examples to validate:

- Client2 cannot fetch Client1 project: GET /projects/1 **Expected:** 403
- Contractor cannot create project **Expected:** 403
- No token **Expected:** 401
- Missing/wrong X-Internal-Key for internal match endpoint **Expected:** 401/403

17.3 Upload security

Try wrong file types and oversized files; confirm 422 validation errors.

17.4 Token revocation test

Delete tokens:

DELETE FROM personal_access_tokens;

Expected: old tokens fail with 401; browser redirects to login on refresh.

18 If Something Breaks (Standard Recovery)

- Reset & reseed:

```
php artisan migrate:fresh then php artisan db:seed --class=TestDataSeeder
```

- Tail logs:

```
tail -f storage/logs/laravel.log
```

- Ensure Reverb running:

```
php artisan reverb:start
```

- Clear caches:

```
php artisan cache:clear php artisan config:clear php artisan route:clear
```

19 Quick Reference | Accounts

admin@test.com	/ Test@1234	-> /dashboard/admin
client@test.com	/ Test@1234	-> /dashboard/client
architect@test.com	/ Test@1234	-> /dashboard/architect (Sara, verified)
contractor@test.com	/ Test@1234	-> /dashboard/contractor (Hassan, verified)
architect2@test.com	/ Test@1234	-> /dashboard/architect (Bilal, verified)
contractor2@test.com	/ Test@1234	-> /dashboard/contractor (Ali, verified)
pending@test.com	/ Test@1234	-> pending verification flow